Approval guidelines

API Documentation

The Stake Development Kit is a comprehensive framework designed to simplify the creation, simulation, and optimization of slot games. Whether you're an independent developer or part of a dedicated studio, the SDK empowers you to bring your gaming vision to life with precision and efficiency. By leveraging the Carrot Remote Gaming Server (RGS), developers can seamlessly integrate their games on Stake.com, facilitating smooth and scalable deployments.

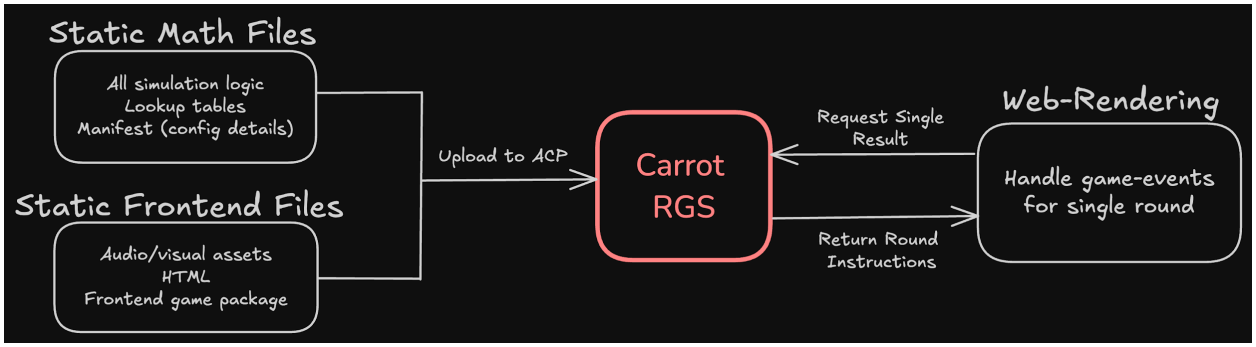
What Does the SDK Offer?

The SDK is an optional software package handling both the client-side rendering of games in-browser, and the generation of static files containing all possible game results.

1. Math Framework: A Python-based engine for defining game rules, simulating outcomes, and optimizing win distributions. It generates all necessary backend and configuration files, lookup tables, and simulation results.
2. Frontend Framework: A PixieJS/Svelte-based toolkit for creating visually engaging slot games. This component integrates seamlessly with the math engine's outputs, ensuring consistency between game logic and player experience.

Stake Engine Game Format Criteria

For verification, testing and security purposes, games uploaded to Stake Engine must consist of static files. Developers utilizing their own frontend and/or math solutions are welcome to upload compatible file-formats to the Admin Control Panel (ACP). All possible game-outcomes must be contained within compressed game-files, typically separated out by modes. Each outcome must be mapped to a corresponding CSV file summarizing a single game-round by a simulation number, probability of selection, and final payout multiplier. When a betting round is initiated a simulation number is selected at a frequency proportional to the simulation weighting, and the corresponding game events are returned though the `/play` API response.



General Requirements

When a game is submitted for approval, Stake Engine technical support will review its suitability for publication on Stake's platform. Approval requests will be actioned for a specific frontend and math version. Our team will inspect the game for functionality, clarity, communication, and technical performance. These factors determine the suitability of your game for publication. Stake Engine will communicate any issues or concerns, providing requested changes and reasoning for approval or rejection on a case-by-case basis.

Approval requests must be accompanied by a short blurb describing your game theme and mechanics for use in promotional material and the game description tag.

Key Restrictions

- Stake Engine games are strictly *stateless*: Each bet must be independent of previous outcomes. Games cannot include jackpots, gamble features, continuation, or early cashout options.
- Team names, game titles, and assets must comply with intellectual property/copyright law. Infringement is grounds for rejection.
- Games must be original designs. Pre-purchased or licensed games existing on other third-party websites will not be permitted.
- Game assets cannot include material with Stake™ branding or themes.
- Approval is at the discretion of the reviewer. Games deemed offensive, explicit, in poor taste, or of insufficient quality may be rejected.
- Games that promote, encourage, or are likely to appeal to underage persons are not permitted. This includes artistic depictions of children or child-like characters in any gambling context.
- Games will be automatically considered for publication on stake.us under the condition that they abide by *strict language requirements* (see *Jurisdiction Requirements*below). Stake Engine offers a *social mode*setting within the play modal to test social languages.

Post Release Notes

Ensure that when submitting a review request, the game is finalized and ready for publication.

Once a game has been approved for publication on Stake/Stake-US, only minor updates to address visual issues are permitted, unless otherwise requested by the Stake Engine team. Changes to the underlying math model, the addition of new game modes, or modifications to gameplay mechanics will not be allowed.

Bet Replay

Overview

Bet Replay is a standard iGaming feature that allows players to view and share the outcome of a round after it has completed. Games must accept a set of query parameters that place the game into replay mode, loading a specific round based on its mode and event ID, along with parameters to configure currency, language, social mode, and bet sizing.

This feature is essential for transparency, player engagement, and support operations.

Approval Requirements

Bet Replay is now a mandatory requirement for all games seeking approval.

New Games

All new games must support Bet Replay. During the game review process, we will:

- Test the replay functionality
- Request a range of event IDs to validate different scenarios
- Games without this feature will not be approved

Existing Games

We strongly encourage existing games to implement this feature as well.

Why Bet Replay?

Benefits for Game Developers

Benefit	Description
Faster Development	Provides a useful interface for front-end development
Better Quality	Test specific scenarios and edge cases
Easier Debugging	Replay rare events that only occur occasionally (e.g., max win screen)
Fewer Bugs	Catch issues before they reach production

Benefits for Operators

Benefit	Description
Bet Queries	Support team can quickly view specific rounds to investigate player issues
Dispute Resolution	See the result exactly as the player did
Bug Investigation	If an event crashes devices, the replay will crash the same way for troubleshooting
Audit Trail	Verify that game rules were followed correctly

Benefits for Players

Benefit	Description
Transparency	Players can verify their results
Social Sharing	Share big wins on social media
Re-watch Wins	Relive exciting moments

Important Notice

PLAYER SESSION IS NOT REQUIRED FOR VIEWING BET REPLAY!

Players can view Bet Replay without an active session or authorization. This means replay URLs can be shared publicly (e.g., on social media, in chat, etc.).

Frontend Integration

1. Query Parameters

Your game will receive the following query parameters when loaded in replay mode:

Parameter	Required	Description

`replay`	Yes	Always `true` when in replay mode
`game`	Yes	Game ID
`version`	Yes	Math version of the game (e.g., `1`, `2`)
`mode`	Yes	Bet mode
`event`	Yes	Unique simulation ID to replay
`rgs_url`	Yes	RGS server URL to fetch replay data from
`currency`	No	Currency code
`amount`	No	Bet amount in units
`lang`	No	Language code

`device`	No	Device type
`social`	No	Social mode (`true`/`false`)

---

2. Fetching Replay Data (RGS Endpoint)

After parsing the query parameters, your game must fetch the replay state from the RGS server.

Endpoint

```
GET {rgs_url}/bet/replay/{game}/{version}/{mode}/{event}
```

Example Request

```
GET
https://rgs.stake-engine.com/bet/replay/01996148-eecf-7678-be46-41de88c58951/1/
SUPER/55
```

Response Schema

```
{
  "payoutMultiplier": 25.0,
  "costMultiplier": 1.0,
  "state": { }
}
```

Field	Type	Description

`payoutMultiplier`	`float`	Multiplier for calculating total payout
`costMultiplier`	`float`	Multiplier for calculating bet cost
`state`	`object`	Game-specific state for replay animation

Expected User Experience

When loading the game in replay mode, follow these UX guidelines:

Loading Phase

1. Auto-load without interaction — The game should load the event data automatically
2. Display “Play” button — Once loaded, show a play button to prompt the user to start the replay

During Replay

1. Play the round as normal — Show all animations, sounds, and visual effects
2. Display results — Show the final outcome exactly as the player saw it
3. No betting allowed — All bet controls must be disabled or hidden

After Replay

1. Show “Play Again” button — Allow users to restart and watch the replay again
2. Display final results — Keep the win amount and outcome visible

UI Simplification

We recommend implementing a slimmed-down UIfor replay mode:

Hide/Remove	Keep/Show
Balance display	Win amount
Play buttons	Replay controls
Bet amount selector	Replay bet amount
Autoplay settings	Currency display

Implementation Checklist

Your game must handle the following in replay mode:

- Detect replay mode — Check for `replay=true` query param
- Fetch replay data — Call the RGS endpoint with correct parameters
- Show loading state — Display a loader while fetching

- Display “Play” button — Prompt the user to start the replay
- Disable betting UI — Hide or disable all bet controls
- Disable session calls — Do not make any authenticated API calls
- Play full animation — Show all animations, sounds, and results
- Show results — Display bet cost, payout, and win amount
- Add “Play Again” button — Allow rewatching the replay
- Handle errors — Show an appropriate message if replay data fails to load
- Prevent session transition — No way to start normal play from replay

---

Testing & Review

During game review, you may be asked to provide event IDs for different scenarios for every bet mode:

- Normal win
- Big win
- Win cap (max win)
- Loss (zero payout)
- Bonus round trigger (if applicable)

Make sure to test edge cases like max wins and rare bonus features before submitting for review.

Game Quality Rankings

All games deemed suitable for publication on Stake Engine receive a Quality Ranking from 0 to 3★'s, where 0 is the lowest and 3 is the highest.
This ranking determines a game's visibility and positioning eligibility.

Ranking Tiers

Rank	Description	Promotion & Visibility
★★★	Awarded only to studio-quality games showing exceptional creativity, uniqueness and attention to detail.	• Optimal positioning and eligible for prominent display in Burst Games, Stake Exclusives, and/or the *featured*section of New Releases.
★★	Given to games that show considerable creativity or originality. While they may lack polish compared to more established studios, they still demonstrate strong development quality and attention to detail.	• Can appear in Burst Games or Stake Exclusives if driven by user popularity. • Placement in New Releases depends on space and demand.
★	Games of lower polish that still meet publishing requirements.	• Published with limited visibility. Always placed at the bottom of New Releases.

		• Not included in promotional categories unless driven by exceptional user demand.
NA	‘0-star’ rating for games that meet technical requirements.	• Limited visibility, can only be found if explicitly searched for. • Not included in promotional or new-release categories.

Review Priority

When a game is submitted for review, it will first receive an initial star rating prior to a comprehensive evaluation. Both new and ongoing reviews are prioritised according to the game’s current star rating.

Category Placement Guidelines (updated weekly)

New Releases

- All games with a 1+ ★ rating will receive a New Release tag.
- Rank 3: Prioritized placement in *featured* and top positioning of New Releases.
- Rank 2: May appear in weekly releases if space allows, otherwise placed lower.
- Rank 1: Always remain at the bottom of New Releases, unless sorted by Newest.

Burst Games

- Priority given to Rank 3 games.
- Rank 2 games may appear in this category if popularity drives demand.

Stake Exclusives

- Priority given to Rank 3 games.
- Rank 2 games may appear within this section if driven by popularity.

Remote Game Server (RGS) Communication

Session authentication and bet transactions are handled exclusively through the Stake Engine RGS. The RGS manages session token generation, *play/*responses, and optional parameters like supported currencies and languages.

RGS Authentication

- Bet Level Verification: The *authenticate* HTTP response returns default bet levels, supported bet levels for a specified currency, and minimum/maximum bet amounts. The frontend must respect these values. Example: If the default bet

size is 1 unit but the session uses JPY (minimum bet size: 10 units), the *play/*request will fail.
- Bet increments must reflect allowed values within *authenticate/config/minStep*.
- Minimum and maximum bet levels must be available for selection as dictated by the RGS.

Cross-Site-Scripting (XSS)

- Stake Engine enforces a strict XSS policy. The game build must consist only of static files and cannot reach external sources. Common issues include downloading fonts from external servers, which logs console errors.

RGS URL

- The game must use the *rgs_url* query parameter to determine the server to call.

Currency and Language

English is the only required language. If only English (en) is supported, on-screen text must not corrupt when other language parameters are passed.

Supported Languages

Language	Abbreviation

Arabic	ar
German	de
English	en
Spanish	es
Finnish	fi
French	fr
Hindi	hi
Indonesian	id
Japanese	ja
Korean	ko
Polish	po
Portuguese	pt
Russian	ru

Turkish	tr
Chinese	zh
Vietnamese	vi

Supported Currencies

Currency	Abbreviation	Display	Example
United States Dollar	USD	$	$10.00
Canadian Dollar	CAD	CA$	CA$10.00
Japanese Yen	JPY	¥	¥10
Euro	EUR	€	€10.00
Russian Ruble	RUB	₽	₽10.00
Chinese Yuan	CNY	CN¥	CN¥10.00
Philippine Peso	PHP	₱	₱10.00
Indian Rupee	INR	₹	₹10.00

Indonesian Rupiah	IDR	Rp	Rp10
South Korean Won	KRW	₩	₩10
Brazilian Real	BRL	R$	R$10.00
Mexican Peso	MXN	MX$	MX$10.00
Danish Krone	DKK	KR	10.00 KR
Polish Złoty	PLN	zł	10.00 zł
Vietnamese Đồng	VND	₫	10 ₫
Turkish Lira	TRY	₺	₺10.00
Chilean Peso	CLP	CLP	10 CLP
Argentine Peso	ARS	ARS	10.00 ARS
Peruvian Sol	PEN	S/	S/10.00
Nigerian Naira	NGN	₦	₦10.00
Saudi Arabia Riyal	SAR	SAR	10.00 SAR

Israel Shekel	ILS	ILS	10.00 ILS
United Arab Emirates Dirham	AED	AED	10.00 AED
Taiwan New Dollar	TWD	NT$	NT$10.00
Norway Krone	NOK	kr	kr10.00
Kuwaiti Dinar	KWD	KD	KD10.00
Jordanian Dinar	JOD	JD	JD10.00
Costa Rica Colon	CRC	₡	₡10.00
Tunisian Dinar	TND	TND	10.00 TND
Singapore Dollar	SGD	SG$	SG$10.00
Malaysia Ringgit	MYR	RM	RM10.00
Oman Rial	OMR	OMR	10.00 OMR
Qatar Riyal	QAR	QAR	10.00 QAR
Bahraini Dinar	BHD	BD	BD10.00

Stake Gold Coin	XGC	GC	10.00 GC
Stake Cash	XSC	SC	10.00 SC

Find code examples for displaying these values at https://stake-engine.com/docs/rgs

RGS Details

This specification outlines the API endpoints available to providers for communicating with the Stake Engine. These APIs enable key operations such as creating bets, completing bets, validating sessions, and retrieving player balances.

Introduction

This document defines how the provider's frontend communicates with the Stake Engine endpoints. It includes a detailed description of the core API functionality, along with the corresponding request and response structures.

The API facilitates communication between your game and the server. Each of the APIs will request the server to perform an action such as; authenticating a session, playing a round of a game and ending a round of a game. The APIs can be here.

Stake Engine NPM Client

Simplify communication to the RGS via the Stake Engine client. This package has helpers to streamline communication with the RGS.

Find information on the package and how to use it here.

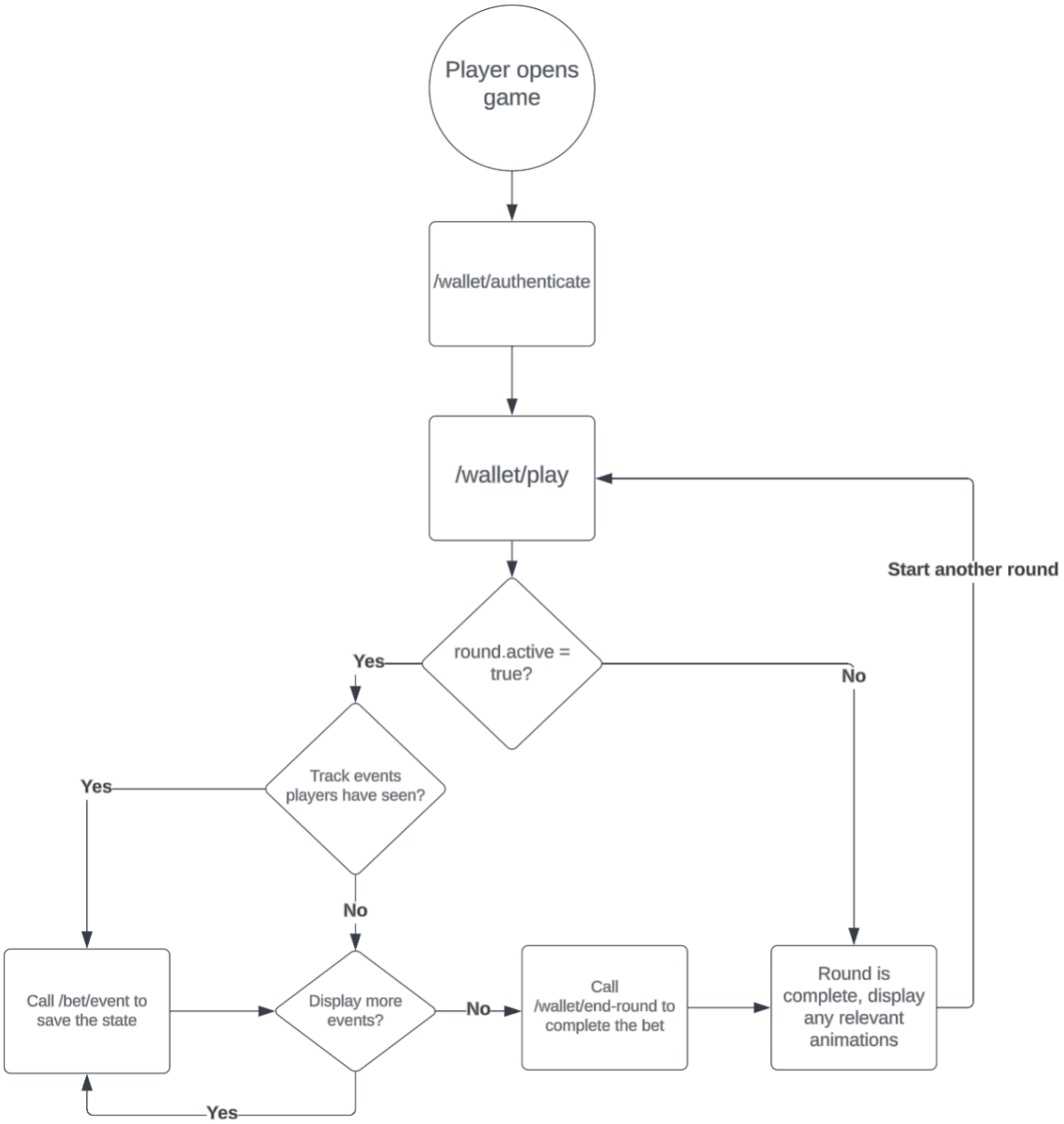
- https://github.com/StakeEngine/ts-client

API flows

All flows require the `/wallet/authenticate`API to be called when the game first loads. This authorizes the sessionID to be used by the `/wallet/play`, `/wallet/balance` and `/wallet/end-round` endpoints. If `/wallet/authenticate` endpoint has not been called with by the game, all subsequent APIs call will be returned with a 400 `ERR_IS` error as the session is invalid.

There the intended way to interact with the Stake Engine RGS API is described as a Basic Flow. This flow takes creates a round and will close the round after all animations have been complete. It accomplishes this by calling the `/wallet/play`API and then calling the `/wallet/end-round`API when the round is complete.

Basic flow



You will call `/wallet/play` and `/wallet/end-round` in a basic flow which is the simplest way to interact with the API. If you have a longer round that may include many steps (such as a bonus round in a slot game) you may want to save where the user is up to watching incase they disconnect. When they reload the game in the future, you can use the value found in `round.event` in the `/wallet/authenticate` API response to know where to display the animations for that round from.

URL Structure

Games are hosted under a predefined URL. Providers should use the parameters below to interact with the RGS on behalf of the user and correctly display game information.

```
https://{{.TeamName}}.cdn.stake-engine.com/{{.GameID}}/{{.GameVersion}}/index.html?sessionID={{.SessionID}}&lang={{.Lang}}&device={{.Device}}&rgs_url={{.RgsUrl}}
```

Query Params in URL

Field	Description
sessionID	Unique session ID for the player. Required for all requests made by the game.
lang	Language in which the game will be displayed.
device	Specifies ‘mobile’ or ‘desktop’.
rgs_url	The URL used for authentication, placing bets, and completing rounds. This URL should not be hardcoded, as it may change dynamically.

Language

The `lang` parameter should be an ISO 639-1language code.

Supported languages:

- `ar` (Arabic)
- `de` (German)
- `en` (English)
- `es` (Spanish)
- `fi` (Finnish)
- `fr` (French)
- `hi` (Hindi)
- `id` (Indonesian)
- `ja` (Japanese)
- `ko` (Korean)
- `pl` (Polish)
- `pt` (Portuguese)
- `ru` (Russian)
- `tr` (Turkish)
- `vi` (Vietnamese)
- `zh` (Chinese)

Understanding Money

Monetary values in the Stake Engine are integers with six decimal places of precision:

Value	Actual Amount
100,000	0.1

1,000,000	1
10,000,000	10
100,000,000	100

For example, to place a $1 bet, pass `"1000000"`as the amount.

Currency impacts only the display layer; it does not affect gameplay logic.

Supported Currencies

Currency	Abbreviation	Display	Example
United States Dollar	USD	$	$10.00
Canadian Dollar	CAD	CA$	CA$10.00
Japanese Yen	JPY	¥	¥10
Euro	EUR	€	€10.00

Russian Ruble	RUB	₽	₽10.00
Chinese Yuan	CNY	CN¥	CN¥10.00
Philippine Peso	PHP	₱	₱10.00
Indian Rupee	INR	₹	₹10.00
Indonesian Rupiah	IDR	Rp	Rp10
South Korean Won	KRW	₩	₩10
Brazilian Real	BRL	R$	R$10.00
Mexican Peso	MXN	MX$	MX$10.00
Danish Krone	DKK	KR	10.00 KR
Polish Złoty	PLN	zł	10.00 zł
Vietnamese Đồng	VND	₫	10 ₫
Turkish Lira	TRY	₺	₺10.00
Chilean Peso	CLP	CLP	10 CLP

Argentine Peso	ARS	ARS	10.00 ARS
Peruvian Sol	PEN	S/	S/10.00
Nigerian Naira	NGN	₦	₦10.00
Saudi Arabia Riyal	SAR	SAR	10.00 SAR
Israel Shekel	ILS	ILS	10.00 ILS
United Arab Emirates Dirham	AED	AED	10.00 AED
Taiwan New Dollar	TWD	NT$	NT$10.00
Norway Krone	NOK	kr	kr10.00
Kuwaiti Dinar	KWD	KD	KD10.00
Jordanian Dinar	JOD	JD	JD10.00
Costa Rica Colon	CRC	₡	₡10.00
Tunisian Dinar	TND	TND	10.00 TND
Singapore Dollar	SGD	SG$	SG$10.00

Malaysia Ringgit	MYR	RM	RM10.00
Oman Rial	OMR	OMR	10.00 OMR
Qatar Riyal	QAR	QAR	10.00 QAR
Bahraini Dinar	BHD	BD	BD10.00
Stake Gold Coin	XGC	GC	10.00 GC
Stake Cash	XSC	SC	10.00 SC

Here are some functions that will help you achieve the display format for the currencies.

```
/**
 * Available currency codes for Stake Engine
 */
type Currency =
  | 'USD' // (United States Dollar)
  | 'CAD' // (Canadian Dollar)
  | 'JPY' // (Japanese Yen)
  | 'EUR' // (Euro)
  | 'RUB' // (Russian Ruble)
  | 'CNY' // (Chinese Yuan)
  | 'PHP' // (Philippine Peso)
  | 'INR' // (Indian Rupee)
  | 'IDR' // (Indonesian Rupiah)
  | 'KRW' // (South Korean Won)
  | 'BRL' // (Brazilian Real)
  | 'MXN' // (Mexican Peso)
  | 'DKK' // (Danish Krone)
  | 'PLN' // (Polish Złoty)
  | 'VND' // (Vietnamese Đồng)
  | 'TRY' // (Turkish Lira)
  | 'CLP' // (Chilean Peso)
  | 'ARS' // (Argentine Peso)
  | 'PEN' // (Peruvian Sol)
  | 'XGC' // Stake US Gold Coin
  | 'XSC'; // Stake US Stake Cash
```

```
/**
 * Currency metadata: symbol, default decimals, symbol placement
 *
 */
const CurrencyMeta: Record<
  Currency,
  { symbol: string; decimals: number; symbolAfter?: boolean }
> = {
  USD: { symbol: '$', decimals: 2 },
  CAD: { symbol: 'CA$', decimals: 2 },
  JPY: { symbol: '¥', decimals: 0 },
  EUR: { symbol: '€', decimals: 2 },
  RUB: { symbol: '₽', decimals: 2 },
  CNY: { symbol: 'CN¥', decimals: 2 },
  PHP: { symbol: '₱', decimals: 2 },
  INR: { symbol: '₹', decimals: 2 },
  IDR: { symbol: 'Rp', decimals: 0 },
  KRW: { symbol: '₩', decimals: 0 },
  BRL: { symbol: 'R$', decimals: 2 },
  MXN: { symbol: 'MX$', decimals: 2 },
  DKK: { symbol: 'KR', decimals: 2, symbolAfter: true },
  PLN: { symbol: 'zł', decimals: 2, symbolAfter: true },
  VND: { symbol: '₫', decimals: 0, symbolAfter: true },
  TRY: { symbol: '₺', decimals: 2 },
  CLP: { symbol: 'CLP', decimals: 0, symbolAfter: true },
  ARS: { symbol: 'ARS', decimals: 2, symbolAfter: true },
  PEN: { symbol: 'S/', decimals: 2, symbolAfter: true },
  XGC: { symbol: 'GC', decimals: 2 },
  XSC: { symbol: 'SC', decimals: 2 },
};

/**
 * Formats a number with its currency symbol, respecting default decimals and
symbol placement.
 * The function is intended to be used for displaying balances.
 */
function DisplayBalance(balance: Balance): string {
  // Grabs the currency, if it doesn't exist in the list then it will display
  // the currency code behind the balance value.
  const meta = CurrencyMeta[balance.currency] ?? {
    symbol: balance.currency,
    decimals: 2,
    symbolAfter: true,
  };
  const formattedAmount = balance.amount.toFixed(meta.decimals);

  if (meta.symbolAfter) {
    return `${formattedAmount} ${meta.symbol}`;
  } else {
    return `${meta.symbol}${formattedAmount}`;
  }
```

```
}
```

Social Casino Currencies

- XGC (Gold)
- XSC (Stake Cash)

Bet Levels

Although bet levels are not mandatory, bets must satisfy these conditions:

1. The bet must fall between `minBet` and `maxBet` (returned from `/wallet/authenticate`).
2. The bet must be divisible by `stepBet`.

It is recommended to use the predefined `betLevels` to guide players.

Example:

```
{
  "minBet": 100000,
  "maxBet": 1000000000,
  "stepBet": 10000,
  "betLevels": [
    100000, // $0.10
    200000,
    400000,
    600000,
    ...
    1000000000 // $1000
  ]
```

```
}
```

Bet Modes / Cost Multipliers

Games may have multiple bet modes defined in the game configuration. Refer to the Math SDK Documentation.

When making a play request:

```
Player debit amount = Base bet amount × Bet mode cost multiplier
```

Response Codes

Stake Engine uses standard HTTP response codes (200, 400, 500) with specific error codes.

400 – Client Errors

Status Code	Description
ERR_VAL	Invalid Request
ERR_IPB	Insufficient Player Balance
ERR_IS	Invalid Session Token / Session Timeout
ERR_ATE	Failed User Authentication / Token Expired
ERR_GLE	Gambling Limits Exceeded
ERR_LOC	Invalid Player Location

500 – Server Errors

Status Code	Description
ERR_GEN	General Server Error
ERR_MAINTENANCE	RGS Under Planned Maintenance

Math Publication File Formats

When publishing math results, ensure that the file-format is abided by. These are strict conditions for successful math file publication.

Frontend and Communication

Frontend checks will include reviewing in-game performance and display to ensure the game is free of visual bugs, has the necessary industry-standard User Interface (UI) components, and behaves as described in the game rules.

Game Communication

Game Display

- Submitted games must use unique audio and visual assets. Assest such as backgrounds, symbols and/or animations provided with the *web-sdk* sample games will not be approved for publication.
- Ensure the game is free of visual bugs, including broken or missing assets or animations.
- Popout view support: Stake offers players to option to use the ‘mini-player’ modal to play games in the background. Games must support this small view without the active game board been visibly distorted.

- The game must support mobile view for commonly used devices, with all UI functionality remaining usable during screen scaling.
- All images and fonts must be loaded from the Stake Engine Content Delivery Network (CDN).

Rules and Paytable

- Game information must be accessible from the UI, including a detailed description of all game rules.
- If multiple game modes are available, provide a description of the cost of each bet and the actions being purchased.
- The RTP of the game (and each mode, if applicable) must be clearly communicated to the player.
- The maximum win amount for each mode must be clearly displayed.
- Payout amounts for all symbol combinations must be presented.
- If the game includes special symbols (e.g., cash prizes or multipliers), list all obtainable values.
- For feature modes (e.g., triggered by Scatter symbols), describe how to access them. Example: “3 Scatters award 10 free spins; 4 Scatters award 15 spins …”

UI Components

- Game must include a User Interface guide, briefly describing the functionality of UI buttons.
- The game must allow players to change the bet size.
- Player must be able to use all bet-levels returned within RGS auth/ response.
- The player’s current balance must be displayed.
- Final win amounts must be clearly shown for non-zero payout results.
- If an outcome contains multiple winning actions, the payout amount must incrementally update to match the final payout multiplier.
- The UI must include an option to disable sounds.
- The spacebar must be mapped to the *bet button*.
- If an ‘autoplay’ feature is present, the player must confirm the autoplay action, games are not allowed to automatically place consecutive bets with one click.

Other Checks

- Check the network tab to ensure no errors or game information is being logged.
- Playtest the game to verify it behaves as described in the rules (e.g., validating payout combinations).
- Game will be tested with various combinations of currencies and languages.
- If the game has a ‘fastplay’ option: wins amounts, winning symbol combinations and pop-up information and must still be legible to player.

Math Verification

Summary statistics and hit-rate tables will be analyzed to ensure the game adheres to industry standards for chance-based casino games and is not misleading.

Summary Statistics

- Verify the mode cost is correctly represented in the game rules for each mode.
- The calculated Return to Player (RTP) must be within 90.0%–98.0%. For multiple modes, all must fall within a 0.5% variation (e.g., base game at 97% RTP requires other modes to be between 96.5% and 97.5%).
- Ensure the maximum win amount matches the description in the game rules for each mode.
- The maximum win must be realistically obtainable (typically more frequent than 1 in 10,000,000, depending on payout amount).
- For slot-type games, run 100,000–1,000,000 simulations to ensure sufficient outcome diversity and avoid repeated results in a single session.
- A reasonable portion of simulations should yield paying results (e.g., 90,000 non-paying results out of 100,000 may be grounds for rejection).
- The hit-rate of the most likely single simulation should not be overwhelmingly dominant if there is a visual expectation that results are sufficiently varied.

Other Considerations

- The hit-rate of non-zero wins should align with industry standards (<1 in 20 bets, or more frequent).
- For “BASE” modes (1x cost), the standard deviation should be within industry norms to ensure reasonable volatility for slot-type games.
- List the number of non-zero weight payouts. Zero-weight payouts should not dominate the provided simulations.
- Inspect hit-rates for win-ranges to avoid gaps where expected win amounts are unobtainable (e.g., intermediate wins should exist between small payouts and the maximum payout amount).

Game Tile Visual Asset Requirements

With each game submission, they must include the submission of visual assets to be used to create the game tile.

It’s important to include high quality, visually appealing assets in order to create a game tile that will appeal to players and entice them to click the game. Games with artwork that look to be of low quality or are visually unappealing often result in lower player trust, lower interest and ultimately lower game engagement.

For the creation of each game tile, we require the following assets:

- Background image
- Foreground image
- Provider Logo

Please ensure that the background & foreground images don't exceed more than 3MB combined.

Background image

- An environmental background that shows the world of the game
- File format: High resolution PNG or JPG file
- Naming convention: `GameTitle-BG.format`(e.g., `CrownConquest-BG.png` or `PixelCastle-BG.jpg`)

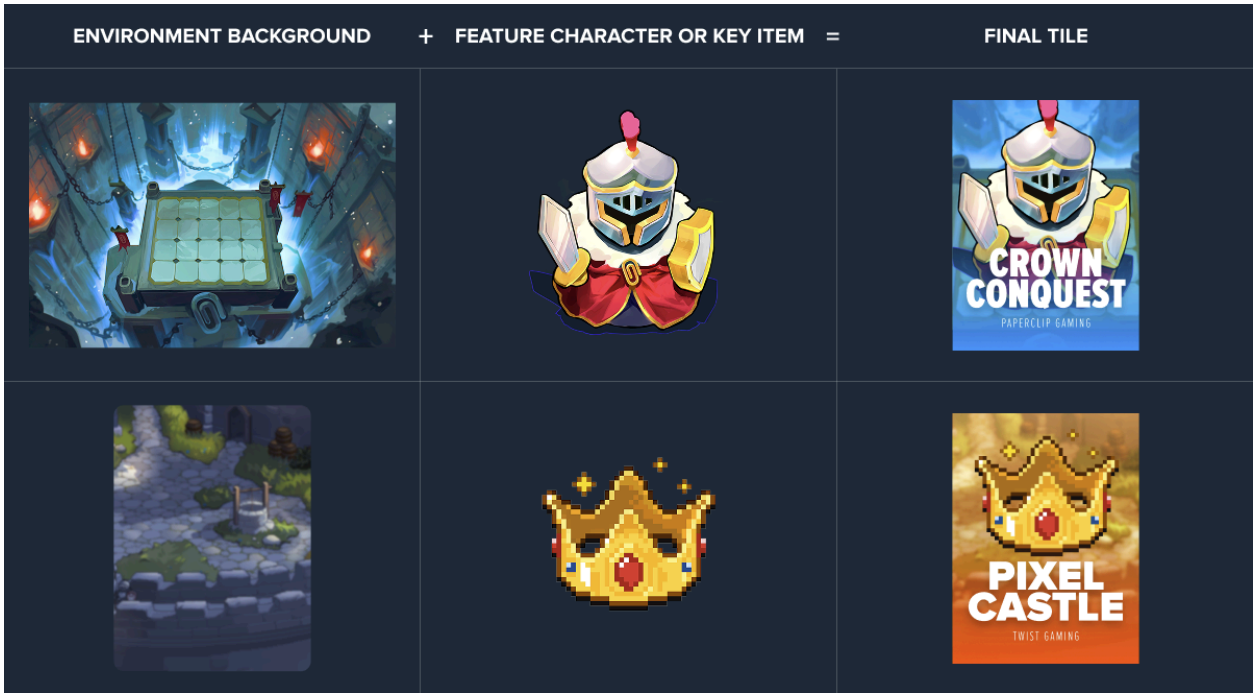
Foreground image

- A feature character or key item that represents the game
- File format: High resolution PNG with a transparent background
- Naming convention: `GameTitle-FG.png`(e.g., `CrownConquest-FG.png`)

Provider Logo

- The official logo of the game provider or studio
- File format: High resolution PNG with a transparent background
- Naming convention: `ProviderName-Logo.png` (e.g., `ZuckGames-Logo.png`)

- Should be clear and legible at small sizes



General Game Disclaimer

The game rules/information popup must include a brief disclaimer regarding game operation. Since Stake Engine utilises pre-calculated game results, payouts are dictated purely by the RGS response and are not influenced by events on the frontend. You are able to use our template disclaimer, or your own, so long as the same message is clearly conveyed.

General Disclaimer:

Malfunction voids all wins and plays. A consistent internet connection is required. In the event of a disconnection, reload the game to finish any uncompleted rounds. The expected return is calculated over many plays. The game display is not representative of any physical device and is for illustrative purposes only. Winnings are settled according to the amount received from the Remote Game Server and not from events within the web browser. TM and © 2026 Stake Engine.

Jurisdiction Requirements

For games to be avaliable on stake.us, US requriements prohibit the use of certain gambling terms. This predominantly applies to game rules but also potentially extends to images and UI elments. For your game to be approved for release on stake.us, your game cannot contain any of the terms listed below.

The RGS uses the URL query parameter `social=true/false` to indicate wheather or not the game is loaded in a ‘social’ casino. We reccomend using an additional language file with the prefix: `sweeps_<lang>` to handle phrase changes.

A table of prohibited terms is given below, along with suggested replacement phrases:

Restricted Phrase	Replacement Phrase
win feature	play feature
pay out	win / won
paid out	win
stake	play amount

pays out	won
betting	play / playing
total bet	total play
bet	play
bets	plays
cash	coins
payer	winner
pay	win
pays	wins
paid	won
money	coins
buy	play
bought	instantly triggered

purchase	play
at the cost of	for
rebet	respin
cost of	can be played for
credit	balance
buy bonus	get bonus
gamble	play
wager	play
deposit	get coins
withdraw	redeem
bonus buy	bonus / feature
be awarded to player’s accounts	appear in player’s accounts
betting	playing

total bet	play
pay out	win / won
paid out	won
place your bets	come and play / join in the game
pays out	win
win feature	play feature
bet/s	play/s
currency	token
fund	balance